4、雷达跟踪趣味玩法

该章节需要搭配小车底盘以及其它传感器才能运行，这里只是说明实现的方式，实际上运行不了，需要搭配本公司的RDK-X3旭日派小车实现该功能。要移植到自己的主板上运行，还需安装其它依赖，因此，这里只是做代码演示，望须知。

4.1、功能说明

程序启动后，雷达扫描最近的一个物体，然后锁定，物体移动，小车跟着移动。

4.2、代码参考路径

该功能源码的位置位于配套虚拟机中，

```
~/oradar_ws/src/yahboomcar_laser/yahboomcar_laser/laser_tracker.py
```

4.3、程序启动

终端输入，

```
ros2 launch yahboomcar_laser laser_tracker_launch.py
```

4.4、核心代码解析

laser_tracker_launch.py

```
from launch import LaunchDescription
from launch_ros.actions import Node

import os
from ament_index_python.packages import get_package_share_directory
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource

def generate_launch_description():
    ms200_scan_node = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(
         get_package_share_directory('oradar_lidar'), 'launch'),
         '/ms200_scan.launch.py'])
      )

    laser_tracker_node = Node(
        package='yahboomcar_laser',
        executable='laser_tracker',
    )

    bringup_node = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(
        get_package_share_directory('yahboomcar_bringup'), 'launch'),
        '/yahboomcar_bringup_launch.py'])
    )
```

```
    launch_description = LaunchDescription([
        ms200_scan_node,
        laser_tracker_node,
        bringup_node
        ])
    return launch_description
```

主要是启动雷达ms200_scan_node、底盘bringup_node和跟踪功能节点laser_tracker_node。主要是看laser_tracker.py文件。

laser_tracker.py

主要看雷达的回调函数，这里说明了如何获取到每个角度的障碍物距离信息，然后找出最近的一个点，接着判断距离，然后就是计算速度数据，最后发布出去，

```
def registerScan(self, scan_data):
        if not isinstance(scan_data, LaserScan): return
        if self.Joy_active == True or self.Switch == False:
            if self.Moving == True:
                self.pub_vel.publish(Twist())
                self.Moving = not self.Moving
            return

        self.Moving = True

        frontDistList = []
        frontDistIDList = []
        minDistList = []
        minDistIDList = []
        ranges = np.array(scan_data.ranges)
        for i in range(len(ranges)):
            angle = (scan_data.angle_min + scan_data.angle_increment * i) * 180
/ pi
            if angle > 180: angle = angle - 360
            if abs(angle) < self.priorityAngle: #priorityAngle是小车优先考虑跟随范围
                if 0 < ranges[i] < self.ResponseDist + self.offset:
                    frontDistList.append(ranges[i])
                    frontDistIDList.append(angle)
            elif abs(angle) < self.LaserAngle and ranges[i] > 0:
                minDistList.append(ranges[i])
                minDistIDList.append(angle)
        #计算出最小距离点及ID
        if len(frontDistIDList) != 0:
            minDist = min(frontDistList)
            minDistID = frontDistIDList[frontDistList.index(minDist)]
        else:
            minDist = min(minDistList)
            minDistID = minDistIDList[minDistList.index(minDist)]

        velocity = Twist()
        #计算线速度
        if abs(minDist - self.ResponseDist) < 0.1:
            velocity.linear.x = 0.0
        else:
```

```
            velocity.linear.x = -self.lin_pid.pid_compute(self.ResponseDist,
minDist)
        #计算角速度
        angle_pid_compute = self.ang_pid.pid_compute(minDistID / 72, 0)
        if abs(angle_pid_compute) < 0.02:
            velocity.angular.z = 0.0
        else:
            velocity.angular.z = angle_pid_compute
        self.pub_vel.publish(velocity)
```